

Project Report:

Community Inventory Alert System

In my local community, many small scale grocery vendors manage their daily inventory using manual paper based logs. This traditional method is highly susceptible to human error, often resulting in stock-outs where essential daily items such as milk, bread, or eggs run out before the shopkeeper realises it. This is a real world computational problem where identifying and abstracting the programming task can lead to a practical and contemporary application. For a small business, these stock outs lead to lost revenue and force community members to travel further for basic needs. By digitizing this process, the shopkeeper can gain knowledge about their stock flow and ensure service reliability.

My approach followed the Top-Down Design as I began analyzing the problem and creating a logic flow that handles data entry, stock calculation, and alert generation. Using Python as the primary language, I utilized the interactive mode to allow the shopkeeper to input sales data in real-time. The core of the solution relies on Python Control Flow, specifically iteration statements like while loops to keep the program running during a shift, and conditionals (if else if) to check if an item's quantity has dropped below a safe threshold.

A critical decision in this project was the choice of data structures. I opted to represent compound data using Python Dictionaries. Using a dictionary allowed me to map item names as "keys" and their quantities as "values," which is significantly more efficient for searching and

updating stock than a standard list. Additionally, I implemented a fixed "Minimum Stock" variable. This architectural choice ensures the code remains clean and scalable, if the shopkeeper decides that a different threshold is needed for certain items, the logic remains consistent while only the parameter changes. This applies the algorithm design techniques learned to evaluate and solve computing problems effectively

One of the primary challenges was ensuring Program Verification and robustness against user error. During initial testing, I found that the program would crash if a user accidentally entered text where a numerical quantity was expected, or if they used different capitalizations (e.g., "Milk" vs "milk"). To solve this, I refined my Python data handling by converting all string inputs to lowercase and implementing basic input validation to ensure variables were treated as the correct types. Another challenge involved handling “duplicates” wherein if a shopkeeper entered the same restock item twice, I had to ensure the algorithm updated the existing count rather than creating a new entry, effectively applying array techniques for removal of duplicates

This project allowed me to successfully read, write, and execute a functional Python program that solves a tangible community issue. The salient feature is the problem solving aspect while a code is simply a tool to execute a well thought out algorithm. Moving from flowcharts and pseudo-code to a live Python script taught me the importance of the flow of execution and parameter handling. Ultimately, this project demonstrated that even fundamental algorithms, when applied purposefully, can create a contemporary application that improves daily life.